🔤 Word Scramble Game

Technical Documentation (Python Edition)

Version 1.0 · Last updated June 2026

Table of contents

1. Introduction

This document describes the internal design and implementation of the Word Scramble Game. It is intended for developers who want to understand, modify, or extend the codebase. For installation and play instructions, see the accompanying README.

The project demonstrates two foundational programming concepts in a self-contained way: string manipulation (converting a word into a list of characters, rearranging them, and joining them back into a string) and randomization (using Python's random module to shuffle letters fairly and select words unpredictably). It deliberately avoids external dependencies so the entire logic is readable in a single file using only the Python standard library.

2. Architecture overview

The game is a single-file command-line application. There is no GUI, database, or network component — all interaction happens through the terminal via input() and print().

Concern	Where it lives
Word data	WORD_LIST constant
Shuffling logic	shuffle_word()
Display / output	show_scoreboard(), print() calls
Round logic	play_round()
Program entry point	main()

Game state is not persisted between runs — closing the program discards the score, attempts, and streak.

3. Data structures

Each word is represented as a dictionary with two keys:

```
{"word": "PLANET", "hint": "Found in space"}
```

These are stored in a module-level list, WORD_LIST. Using a list of dictionaries (rather than two parallel lists) keeps each word and its hint bound together, which avoids index-mismatch bugs if the list is reordered or edited.

4. Core algorithm — Fisher-Yates shuffle

The letter-scrambling logic relies on Python's built-in random.shuffle(), which implements the Fisher-Yates shuffle internally:

```
def shuffle_word(word: str) -> str:
  letters = list(word)        # "PLANET" -&gt; ['P','L','A','N','E','T']
  random.shuffle(letters)     # shuffled in place
  scrambled = "".join(letters)

  if scrambled == word:
    return shuffle_word(word)
  return scrambled
```

Why Fisher-Yates matters: it guarantees every possible ordering of the letters is equally likely. Naive approaches — like swapping two random positions a fixed number of times — can produce biased results where some letter arrangements appear more often than others.

Why the recursive re-shuffle check: if the shuffled result happens to be identical to the original word (possible with short or repetitive words), the function calls itself again so the player is never shown an "unscrambled" word.

Known edge case: a one-letter word can never produce a different arrangement, so this function would recurse indefinitely on a single character. The current WORD_LIST avoids this by using words of 5+ letters.

5. Function reference

Function	Signature	Description
`shuffle_word`	shuffle_word(word: str) -> str	Returns a randomly scrambled version of word
`show_scoreboard`	show_scoreboard(score, attempts, streak) -> None	Prints the current score, attempts, and streak
`play_round`	play_round(entry, score, attempts, streak) -> tuple	Runs one round of guessing for a single word
`main`	main() -> None	Program entry point; runs the full game loop

6. Game state & data flow

Game state is passed explicitly between functions as plain values (score, attempts, streak) rather than stored in global variables or a class. This keeps the control flow easy to trace for beginners, at the cost of slightly more verbose function signatures.

Round lifecycle:

- main() shuffles a copy of WORD_LIST so words aren't repeated in the same order every run, then loops over each entry
- For each word, play_round() is called with the current state
- Inside play_round(), the word is scrambled once via shuffle_word() and shown to the player

- The function loops on input() until the player types the correct word, HINT, or QUIT
- The updated (score, attempts, streak, keep_playing) tuple is returned to main(), which either continues to the next word or ends the game

```
main()
&amp;nbsp;&amp;nbsp;└─ loop over shuffled WORD_LIST
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;└─ play_round(entry, score, attempts, streak)
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;├─ shuffle_word(word)
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;├─ input loop: HINT / QUIT / guess
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;└─ return (score, attempts, streak, keep_
playing)
```

7. Testing

Unit tests live in tests/test_word_scramble.py and use Python's built-in unittest framework — no extra dependencies required. Run all tests with:

```
python -m unittest discover tests
```

What's covered:

- shuffle_word() preserves the original letters (same multiset)
- shuffle_word() preserves word length
- shuffle_word() never returns the original word unchanged
- WORD_LIST is non-empty and every entry has both a word and hint key
- All words in WORD_LIST are stored in uppercase

What's not covered: the interactive play_round() and main() functions are not unit-tested directly since they depend on input(). A future improvement could refactor input handling behind an injectable interface to make these testable too.

8. Extending the game

Add more words

```
WORD_LIST.append({"word": "PYTHON", "hint": "A popular programming language"})
```

Add a timer

Wrap the input() call in play_round() with a timeout mechanism (e.g. using the threading or signal module on Unix-like systems) to limit how long a player has to answer.

Add difficulty levels

Split WORD_LIST into multiple lists grouped by word length or topic, and let the player choose one in main() before the round loop begins.

Persist high scores

Write the final score to a local file (e.g. scores.txt or a small JSON file) at the end of main(), and read it back on startup to display a "best score" alongside the current run.

Make input testable

Extract the input() calls in play_round() behind a small wrapper function so tests can inject predetermined input sequences without using real stdin.

9. Known limitations & future work

- No persistent storage — score, attempts, and streak reset every run
- shuffle_word() would recurse indefinitely on a single-letter word (not currently an issue since all words in WORD_LIST are 5+ letters)
- play_round() and main() are not directly unit-tested due to their reliance on input()
- No difficulty levels — all words are presented with equal probability
- No support for word categories or themed rounds

End of technical documentation.